

MySQL应用教程

第七章 MySQL索引

概述

- ▶ 在MySQL中，索引其实与书的目录非常的相似，由数据表中一列或多列组合而成，创建索引的目的是为了优化数据库的查询速度，提高性能的最常用的工具。
- ▶ 所有MySQL列类型都可以被索引，对相关列使用索引是提高select操作性能的最佳途径。
- ▶ 索引有两种存储类型：B型树（BTREE）索引和哈希（HASH）索引。其中B型树为系统默认索引存储类型。

内容提要

- 索引的分类
- 索引的创建与管理
- B+TREE索引的左匹配原则



1 索引的分类

普通索引： 在创建普通索引时，不附加任何限制条件。

唯一性索引： 使用UNIQUE参数可以设置索引为唯一性索引。

全文索引： 使用FULLTEXT参数可以设置索引为全文索引。

单列索引： 在表中的单个字段上创建索引。

多列索引： 多列索引是在表的多个字段上创建一个索引。

2 索引的定义与管理——创建索引

间接创建索引

- 通过定义主键约束或者唯一性键约束和外键约束，也可以间接创建索引。
- 主键约束或者唯一性键约束创建的索引的优先级高于使用CREATE INDEX语句创建的索引。

2 索引的定义与管理——创建索引

直接创建索引，有以下三种方式

(1) 在创建表的时候创建索引。

```
create table table_name  
(  
    属性名, 数据类型 [完整性约束],  
    属性名, 数据类型 [完整性约束],  
    .  
    .  
    .  
    属性名, 数据类型 [完整性约束],  
    index | key [索引名] ( 属性名 [ ( 长度 ) ]  
    [ asc | desc ] )  
);
```

2 索引的定义与管理——创建索引

(2) 在已存在的表上创建索引

```
create index 索引名 ON 表名 (属性名 [ ( 长度) ]  
[ asc | desc ]);
```

(3) 使用alter table语句来创建索引。

```
alter table table_name  
add index | key [索引名] ( 属性名 [ ( 长度 ) ]  
[ asc | desc ])
```

2 索引的定义与管理——查看索引

语法格式：

```
INDEX FROM table_name [FROM db_name]
```

另一种语法格式：

```
SHOW INDEX FROM mytable FROM mydb;
```

```
SHOW INDEX FROM mydb.mytable;
```



各个参数的含义：

Table: users	(表名称)
Non_unique: 0	(索引能不能包括重复词, 不能为0, 能则为1)
Key_name: PRIMARY	(索引名称)
Seq_in_index: 1	(索引中的列序号, 从1开始)
Column_name: id	(列名称)
Collation: A	(列以什么方式存储在索引中。A表示升序或NULL无分类)
Cardinality: 6	(索引中唯一值的数目的估计值)
Sub_part: NULL	(如果列只是部分地被编入索引, 则为被编入索引的字符的数目)
Packed: NULL	(指关键字如何被压缩, 如果没有被压缩, 则为NULL)
Null:	(如果列含有null, 则显示YES, 否则不显示)
Index_type: BTREE	(索引类型: BTREE, HASH)
Comment:	(评注)
Index_comment:	

2 索引的定义与管理——删除索引

语法格式：

```
drop index index_name on table_name;
```

```
alter table table_name drop index
```

```
index_name;
```

注意：如果从表中删除某列，则索引会受影响。对于多列组合的索引，如果删除其中的某列，则该列也会从索引中删除。如果删除组成索引的所有列，则整个索引将被删除。

3 索引左匹配原则

- ▶ **A: 全值匹配**
 - ▶ 全值匹配指的是和索引中的所有列进行匹配，即可用于查找姓名和出生日期
- ▶ **B: 匹配最左前缀**
 - ▶ 如：只查找姓，即只使用索引的第一列
- ▶ **C: 匹配列前缀**
 - ▶ 也可以只匹配某一列值的开头部分，如：匹配以J开头的姓的人，这里也只是使用了索引的第一列，且是第一列的一部分
- ▶ **D: 匹配范围值**
 - ▶ 如查找姓在allen和barrymore之间的人，这里也只使用了索引的第一列
- ▶ **E: 精确匹配某一列并范围匹配另外一列**
 - ▶ 如查找所有姓为allen，并且名字字母是K开头的，即，第一列last_name精确匹配，第二列first_name范围匹配
- ▶ **F: 只访问索引的查询**
 - ▶ btree通常可以支持只访问索引的查询，即查询只需要访问索引，而无需访问数据行，即，这个就是覆盖索引的概念。需要访问的数据直接从索引中取得。

```
例create table people(  
last_name varchar(50) not null,  
first_name varchar(50) not null,  
age int not null,gender enum('m','f') not null,  
index ln_fn_a(last_name,first_name,age));
```